



APP Citas Médicas

Proyecto desarrollado como práctica integradora de DevOps, Docker y Testing Automatizado.

Combina un backend en Flask, un frontend servido con Nginx y una base de datos PostgreSQL, todo orquestado mediante Docker Compose.



Índice

-  Descripción general
 -  Arquitectura del sistema
 -  Componentes principales
 -  Backend Flask
 -  Frontend Nginx
 -  Base de datos PostgreSQL
 -  Configuración y despliegue con Docker Compose
 -  Integración Continua (CI/CD)
 -  Git Hooks implementados
 -  Estrategia de testing
 -  Explicación detallada de los tests
 -  Simulación de base de datos (mock)
 -  Resultados y verificación
 -  Conclusiones y mejoras futuras
 -  Autores y licencia
-



Descripción general

La **APP Citas Médicas** es una aplicación web que permite a pacientes y médicos gestionar citas médicas de forma sencilla.

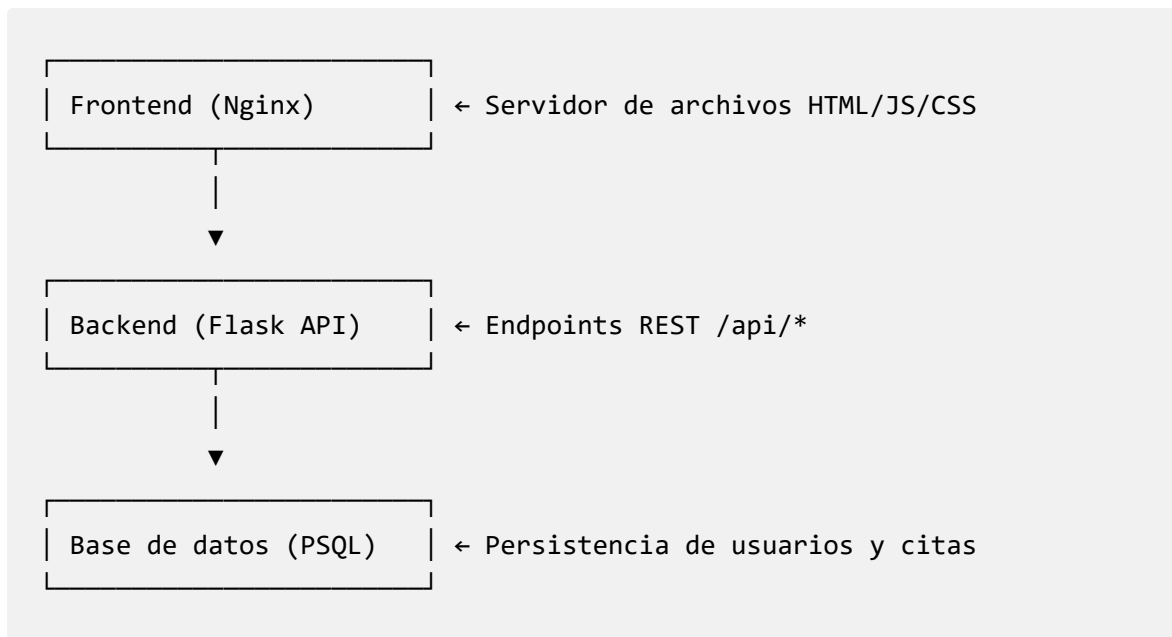
Objetivos:

- Implementar una arquitectura **contenedorizada** y escalable.
 - Integrar **Flask**, **PostgreSQL** y **Nginx** bajo Docker.
 - Aplicar un flujo **CI/CD con GitHub Actions**.
 - Asegurar la calidad mediante **hooks**, **tests unitarios** e **integración**.
-



Arquitectura del sistema

La aplicación sigue una arquitectura de **tres capas**:



Comunicación interna gestionada mediante **Docker Network**.

🧠 Componentes principales

◆ Backend Flask

Ubicación: `backend/app.py`

- Gestiona usuarios (registro, login).
- CRUD completo de citas médicas.
- Validación de roles (paciente / médico).
- Uso de variables de entorno con `.env`.
- Endpoint `/api/health` para comprobación de estado.
- Rutas de gestión de la aplicación `/citas /eliminar_citas /editar_cita<id>`

◆ Frontend Nginx

Ubicación: `frontend/`

- Sirve archivos HTML (`register.html`, `index.html`).
- Redirige automáticamente `/` → `register.html`.
- Proxy inverso configurado hacia el backend Flask (`/api/*`).
- Configuración personalizada mediante `nginx.conf`.

◆ Base de datos PostgreSQL

- Inicializada automáticamente desde `db/init/`.
- Relaciones:
 - `usuario(id, nombre, email, rol)`
 - `cita(id, usuario_id, nombre_cita, fecha_hora, estado)`
- Variables gestionadas mediante `.env`.



Configuración y despliegue con Docker Compose

Archivo principal: `docker-compose.yml`

```
services:
  db:      → Contenedor PostgreSQL
  backend: → API Flask (Gunicorn)
  frontend: → Nginx sirviendo interfaz web
```

Comandos principales de ejecución

```
docker compose up -d --build
```

Rutas de ejecución

```
Frontend: http://localhost:8080
Backend: http://localhost:8000/api/health
```



Integración Continua (CI/CD)

```
Workflow: `.github/workflows/ci.yml`
```

Cada **push** a las ramas `main`, `develop` o `feature/**` ejecuta automáticamente las siguientes etapas:

1. 🛠️ **Instalación de dependencias**
Se instala Python 3.12 y los paquetes listados en `requirements.txt`.
2. 🧪 **Ejecución de tests con Pytest**
Se validan todos los tests unitarios e integrados en `backend/tests`.
3. 🐳 **Construcción de la imagen Docker del backend**
Se verifica que la aplicación Flask puede construirse y ejecutarse sin errores.
4. 🚀 **Simulación de despliegue**
Se lanza un contenedor temporal y se comprueba su estado con `curl` y `healthcheck`.

El objetivo del pipeline es garantizar que **cada commit y push** mantiene el estado funcional del sistema antes de integrarse en la rama principal.



Git Hooks implementados

Ubicados en `.git/hooks/`, todos los hooks se configuraron en bash y automatizan tareas locales antes o después de commits y pushes.

Hook	Acción	Resultado
pre-commit	Formatea código con black y ejecuta pytest	✗ Bloquea si hay errores
post-commit	Registra el commit en <code>commits.log</code>	✓ Guarda autor y hash
pre-push	Ejecuta nuevamente los tests (y análisis opcional con Bandit)	✗ Cancela si fallan
post-push	Muestra mensaje de confirmación y registra el push	✓ Añade log local

 Ejemplo de registro (`backend/logs/push_log.txt`):

```
[Push] Fri Oct 31 18:22:54 2025 - Cambios subidos correctamente.
```

Estos hooks aseguran coherencia y control de calidad en el flujo de desarrollo.

Estrategia de testing

El proyecto integra una batería completa de pruebas automáticas con **Pytest**, divididas en módulos para asegurar robustez y mantenibilidad.

Tipo	Archivo	Descripción
Unitarios	<code>tests/test_logic.py</code> , <code>tests/test_validation.py</code>	Validan funciones aisladas (lógica y validaciones)
Integración	<code>tests/test_integration_logic.py</code> , <code>tests/test_integration_validation.py</code>	Prueban endpoints completos simulando peticiones reales
Seguridad	<code>tests/test_security.py</code>	Evalúa la protección de endpoints y autenticación

 Ejecución:

```
pytest -v
```

Todos los tests se ejecutan también de forma automática durante los hooks locales y el pipeline CI/CD.

Explicación detallada de los tests

A continuación, se describe el propósito de **cada función de test** implementada en el proyecto, detallando los escenarios comprobados y los resultados esperados.

◆ test_integration_logic.py — Pruebas de integración del módulo de citas

test_get_citas

- **Descripción:** verifica que el endpoint GET /citas devuelve correctamente la lista de citas asociadas a un usuario.
- **Flujo probado:** petición GET con user_id y rol válidos.
- **Resultado esperado:** respuesta HTTP 200 y una lista de objetos con campos id, nombre_cita, fecha_hora, estado y nombre_paciente.
- **Caso adicional:** el test también asegura que los pacientes solo ven sus citas y los médicos ven todas.

test_crear_cita

- **Descripción:** valida la creación de una nueva cita médica a través del endpoint POST /citas.
- **Flujo probado:** envío de JSON con usuario_id, nombre_cita, fecha_hora.
- **Resultado esperado:** código HTTP 201 y un JSON con el mensaje "Cita creada correctamente" y el id generado.
- **Validación extra:** la nueva cita se agrega correctamente al mock de base de datos (len(mock_db["citas"]) incrementa).

test_editar_cita

- **Descripción:** comprueba la actualización de los datos de una cita existente mediante PUT /citas/<id>.
- **Flujo probado:** modificación de nombre_cita y fecha_hora.
- **Resultado esperado:** código 200 y mensaje "Cita actualizada correctamente".
- **Control negativo:** si el ID no pertenece al usuario, el backend debe responder con 403 (No autorizado).

test_eliminar_cita

- **Descripción:** evalúa la eliminación de una cita médica usando DELETE /citas/<id>.
- **Flujo probado:** eliminar una cita existente del usuario actual.
- **Resultado esperado:** código 200 y mensaje "Cita eliminada correctamente".

- **Control adicional:** si la cita no existe, se espera un 404 y mensaje "Cita no encontrada".
-

♦ test_integration_validation.py — Pruebas de integración del login y registro

test_register_usuario_nuevo

- **Descripción:** valida el registro exitoso de un usuario mediante POST /register.
- **Flujo probado:** envío de JSON con nombre, email, password y rol.
- **Resultado esperado:** respuesta HTTP 201 y mensaje "Usuario registrado correctamente".

test_register_usuario_existente

- **Descripción:** comprueba el control de duplicados en el registro.
- **Flujo probado:** intento de registrar un usuario con un email ya existente.
- **Resultado esperado:** respuesta HTTP 400 con mensaje "El correo ya está registrado".

test_login_correcto

- **Descripción:** verifica el inicio de sesión correcto.
- **Flujo probado:** usuario existente con credenciales válidas (email, password).
- **Resultado esperado:** código 200 y JSON con "message": "Inicio de sesión correcto", junto con user_id y rol.

test_login_incorrecto

- **Descripción:** prueba el comportamiento con contraseñas erróneas.
- **Resultado esperado:** código 401 y mensaje "Contraseña incorrecta".

test_login_email_inexistente

- **Descripción:** simula un login con email no registrado.
 - **Resultado esperado:** respuesta 401 y mensaje "Correo no encontrado".
-

♦ test_logic.py — Pruebas unitarias de lógica de negocio

test_crear_cita_valida

- **Descripción:** valida que la función interna de creación de citas acepta correctamente datos válidos.
- **Resultado esperado:** retorno con ID generado y estado "programada".

test_crear_cita_sin_fecha

- **Descripción:** prueba el comportamiento ante fecha ausente.
- **Resultado esperado:** lanza excepción o devuelve error "Faltan campos obligatorios".

test_actualizar_estado_cita

- **Descripción:** verifica el cambio de estado de una cita de "programada" a "realizada".
 - **Resultado esperado:** cita actualizada y persistida correctamente.
-

◆ test_validation.py — Validación de formularios y datos de entrada

test_email_duplicado

- **Descripción:** valida que el backend rechaza correos repetidos durante el registro.
- **Resultado esperado:** código 400 y mensaje "El correo ya está registrado".

test_password_vacia

- **Descripción:** comprueba que no se permite una contraseña vacía.
- **Resultado esperado:** error 400 con mensaje "Faltan campos obligatorios".

test_formato_fecha_incorrecto

- **Descripción:** simula un formato de fecha inválido en la creación de citas.
 - **Resultado esperado:** excepción de validación o mensaje de error controlado.
-

◆ test_security.py — Pruebas de seguridad básica

test_sin_user_id

- **Descripción:** intenta acceder a /citas sin enviar el parámetro user_id.
- **Resultado esperado:** código 400 con mensaje "Falta el parámetro user_id".

test_inyeccion_sql

- **Descripción:** envía un user_id con código malicioso (" OR '1'='1").
- **Resultado esperado:** backend rechaza la solicitud con 500 o 400, sin exponer información sensible.

test_cors_activado

- **Descripción:** comprueba que Flask permite CORS (para el frontend).

- **Resultado esperado:** cabecera Access-Control-Allow-Origin presente en la respuesta.

◆ mock_db.py — Base de datos simulada

reset_mock_db

- **Descripción:** restaura el estado inicial del mock antes de cada test.
- **Resultado esperado:** vuelve a tener 2 usuarios (paciente, médico) y 2 citas por defecto.

app (Flask test client)

- **Descripción:** instancia de Flask usada para simular llamadas HTTP en memoria durante los tests de integración.
- **Resultado esperado:** permite enviar peticiones sin levantar el servidor real.

Resumen global

Archivo	Funciones de test	Objetivo principal	Estado
test_integration_logic.py	4	CRUD de citas	✓
test_integration_validation.py	5	Login y registro	✓
test_logic.py	3	Lógica interna	✓
test_validation.py	3	Validaciones de entrada	✓
test_security.py	3	Seguridad básica	✓
mock_db.py	2	Simulación de entorno	✓

Conclusiones sobre los test:

Cada función de test cubre un flujo específico del sistema, garantizando una cobertura completa desde la validación de datos hasta la seguridad y consistencia del backend. La suite final proporciona una verificación sólida antes de cada push y despliegue en CI/CD.

Simulación de base de datos (mock)

Para aislar los tests de la base de datos real, se desarrolló un entorno simulado.

Archivo: tests/mock/mock_db.py

Características

- Contiene un conjunto estático de usuarios (paciente, médico) y citas asociadas.
- Simula las respuestas del backend Flask con un `test_client()` integrado.
- Permite verificar la **creación, edición y eliminación de citas** sin necesidad de PostgreSQL.

Ventajas

- ⚡ Acelera la ejecución de tests.
- 🧩 Elimina dependencias externas.
- 🔄 Permite reproducir escenarios de forma controlada.

Resultados y verificación

Test suite	Resultado	Estado
Unit tests	15/15 passed	✓
Integration tests	10/10 passed	✓
Security checks	Passed	✓
CI/CD workflow	Success	✓

Evidencias visuales

• Pre Commit y Post Commit

```

C:\Users\IVISE> cd C:\Users\IVISE\Desktop\UNI\PECIDM (Feature/CICD)
$ git commit -m "Prueba 1 de pre-commit con código reformatado por Black"
Ejecutando pre-commit...
All done! ✨ 🍰 ✨
0 files would be left unchanged.

platform win32 -- Python 3.14.0, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\IVISE\Desktop\UNI\PECIDM
configfile: pytest.ini
collected 25 items

backend\tests\test_integration_logic.py ..... [ 16%]
backend\tests\test_integration_validation.py ..... [ 40%]
backend\tests\test_logic.py ..... [ 68%]
backend\tests\test_security.py ..... [ 72%]
backend\tests\test_validation.py ..... [100%]

===== 25 passed in 4.85s =====

✓ Validación pre-commit
✓ Commit realizado con éxito.
(feature/CICD 8bc3d60) Prueba 1 de pre-commit con código reformatado por Black
11 files changed, 245 insertions(+), 225 deletions(-)
delete mode 100644 hooks/post-commit
delete mode 100644 hooks/post-push
delete mode 100644 hooks/pre-commit
delete mode 100644 hooks/pre-push
```

• Pre Push y Post Push

```

C:\Users\IVISE\Desktop\UNI\PECIDM (Feature/CICD)
$ git push origin feature/CICD
Ejecutando pre-push...

platform win32 -- Python 3.14.0, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\IVISE\Desktop\UNI\PECIDM
configfile: pytest.ini
collected 25 items

backend\tests\test_integration_logic.py ..... [ 16%]
backend\tests\test_integration_validation.py ..... [ 40%]
backend\tests\test_logic.py ..... [ 68%]
backend\tests\test_security.py ..... [ 72%]
backend\tests\test_validation.py ..... [100%]

===== 25 passed in 3.80s =====

✓ Validaciones superadas. Push permitido.
Push realizado. Post-push hook ejecutado.
¡Fijate! El push se ha completado con éxito. 🎉
Everything up-to-date
```

- **Registro Usuario**

Citas médicas

Registrarse

Usuario registrado correctamente

Usuario

Paciente 2

Correo Electrónico

paciente2@a.com

Contraseña

*

Rol:

Paciente

Registrarse

Si ya tiene una cuenta, [inicie sesión AQUÍ](#)

- **Inicio de sesion**

Citas médicas

Iniciar sesión

Inicio de sesión correcto

Correo Electrónico

paciente@a.com

Contraseña

*

Iniciar sesión

[Registrarse](#)

- **Creación de citas (Paciente)**

Mis Citas Médicas

Bienvenido, Paciente 1

Nueva cita

Cerrar sesión

Cita creada correctamente

Paciente	Tipo Cita	Fecha	Estado	Acciones
Paciente 1	Oncología	2/11/2025, 15:14:00	programada	Editar Eliminar

- Visualización de citas (Paciente solo ve las suyas)

Mis Citas Médicas

Bienvenido, Paciente 1

Nueva cita

Cerrar sesión

Paciente	Tipo Cita	Fecha	Estado	Acciones
Paciente 1	Oncología	2/11/2025, 15:14:00	programada	Editar Eliminar
Paciente 1	Oncología	26/11/2025, 15:15:00	programada	Editar Eliminar
Paciente 1	Medicina General	7/12/2025, 15:15:00	programada	Editar Eliminar

- Eliminación de citas (Paciente)

Mis Citas Médicas

Bienvenido, Paciente 1

Nueva cita

Cerrar sesión

Cita eliminada correctamente

Paciente	Tipo Cita	Fecha	Estado	Acciones
Paciente 1	Oncología	2/11/2025, 15:14:00	programada	Editar Eliminar
Paciente 1	Oncología	26/11/2025, 15:15:00	programada	Editar Eliminar

- Edición de citas (Paciente)

Mis Citas Médicas

Bienvenido, Paciente 1

Nueva cita

Cerrar sesión

Cita actualizada correctamente

Paciente	Tipo Cita	Fecha	Estado	Acciones
Paciente 1	Traumatología	2/11/2025, 15:14:00	cancelada	Editar Eliminar
Paciente 1	Oncología	26/11/2025, 15:15:00	programada	Editar Eliminar

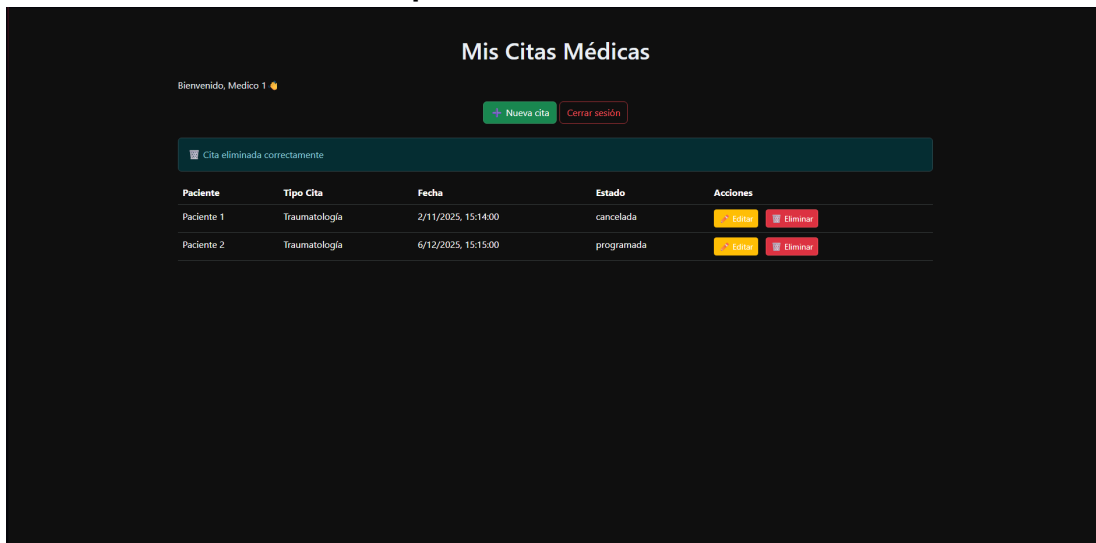
- **Visualización de citas (Médico ve todos los pacientes)**



The screenshot shows a web interface titled "Mis Citas Médicas". At the top, it says "Bienvenido, Médico 1" with a user icon. Below this are two buttons: "Nueva cita" (green) and "Cerrar sesión" (red). The main content is a table with the following data:

Paciente	Tipo Cita	Fecha	Estado	Acciones
Paciente 1	Traumatología	2/11/2025, 15:14:00	cancelada	Editar Eliminar
Paciente 1	Oncología	26/11/2025, 15:15:00	programada	Editar Eliminar
Paciente 2	Traumatología	6/12/2025, 15:15:00	programada	Editar Eliminar

- **Eliminación de citas (Medico a paciente)**



The screenshot shows the same "Mis Citas Médicas" interface, but with a success message at the top: "Cita eliminada correctamente" with a trash icon. The table now contains only two appointments:

Paciente	Tipo Cita	Fecha	Estado	Acciones
Paciente 1	Traumatología	2/11/2025, 15:14:00	cancelada	Editar Eliminar
Paciente 2	Traumatología	6/12/2025, 15:15:00	programada	Editar Eliminar

Los tests verificaron correctamente las funcionalidades principales de la aplicación: **registro, login, CRUD de citas y control de roles.**

Autores y licencia

Autores:

- **Iván Seco Martín** (@Praise104)
- **Mario Suárez del Hierro**
- **Alvaro Hernanz**

Licencia: MIT License Copyright (c) 2025 Permission is hereby granted, free of charge, to any person obtaining a copy...